

Theory of Computation

By VOID
@i.m.void
PAGE NO :
DATE :

Hand written Notes

(TOC)
All Modules

- Module 01 - Why Study TOC, what is String & language
Module 02 - Regular Expression
Module 03 - Context free Grammars (CFG)
Module 04 - Context-free & Non-context-free languages.
Module 05 - Variants of Turing Model (TM)

Will try Theory then Problems

How to study?

Just Read this damn notes damn it
It contains all shit.

→ Module 01

→ Concept of Automata Theory -

1. Alphabet - An non empty Set of Symbols and Symbols can be anything from a-z, 0-9 or special characters
2. String - An finite Set of Symbols from Alphabet (Σ)
3. Concatenation of String - The Concatenation of string u and v is string obtained by writing elements of u followed by elements of v
4. Substring - Strings which are obtained from the universal string (Σ)
5. Prefix - A Prefix of String of any no of leading Symbols
6. Suffix - Suffix is any no of trailing Symbols

$$\begin{aligned} \Sigma^* &= \Sigma^+ + \epsilon \\ \Sigma^+ &= \Sigma^* - \epsilon \end{aligned}$$

PAGE NO.:

DATE:

7. Reversal of a String - The reversal of a string is obtained by writing the string in reverse. (*)
8. length of a String - The length of a string is the no of symbols in it and denoted by $|w|$. Ans
9. Power of the alphabet - the Power of the alphabet is denoted by Σ^i is the set of words length i .
10. clean closure (Σ^*) is a set of symbols with start with ϵ or zero.
11. Clean Star (Σ^+) is a set of symbols which start without zero or ϵ .
12. language - A language can be defined as set of strings obtained from Σ^* where Σ is the alphabet set. →
13. Sentence - A string that belongs to language is called sentence / word. (*)

Different types of language

- Type 0 (Phrase Structured language)
- Type 1 (Context Sensitive language)
- Type 2 (Context free language)
- Type 3 (Regular language)

(*) Regular language is a language accepted by DFA, NFA & ENFA. Ans -

(*) Abstract Machine - It's a (Conceptual or theoretical) Model for Computer hardware (or) Software System, which really do not exist.

Q1 What is finite Automata? (FA)

Ans - A finite Automata is an Mathematical Model Made for studying Abstract automata. On feeding an string the Machine may accept or reject the string. Using this Model the Actual Behaviour of the Model can be understood & build to perform various activities.

It has 3 components:

(*) Input Unit

(*) Control Unit

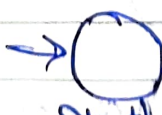
(*) Output Unit

→ Working - (*) Machine assumes to be in Start State.
 (x) The Input Pointer points to 1st cell of the tape.
 (x) After Scanning the current State the pointer goes to next State.
 (x) On reaching the final State the Machine Says if the string is Accepted or Rejected.

FA & ADFA can have One Starting State and Multiple Ending (final State).



Final state



Starting state



Self loop



next state transition

Q2 What is an DFA? (Deterministic Finite Automata)?

Ans - A DFA is a five tuple Machine

$$M = (Q, \Sigma, \delta, q_0, F)$$

M = Machine
 Q = finite non-empty set
 Σ = non-empty finite input alphabet
 $\delta: Q \times \Sigma \rightarrow Q$ (Transition State)
 q_0 : Starting state ($q_0 \in Q$)
 $\circ$: final state

PAGE NO.:

DATE:

An DFA Must Process all Input Symbols, Can have only 1 Starting state, the Transition State Must be Predictable

Both ENFA & NFA Must be Converted to DFA in order to design a Real life Component Ex- Compiler

DFA Can be considered for

- (X) Pattern Recognition Problem
- (X) Divide by K problems
- (X) Modular K-problem

Q3 - What is an NFA? (Non-Deterministic FA)

Ans - A Simple FA where it contains 5 components

$M = (Q, \Sigma, \delta, q_0, F)$

M : Name of Machine

Q = Non-Empty finite set

Σ = $Q \times \Sigma = Q$ (non-empty finite set of alphabet)

q_0 = $q_0 \in Q$ (Starting state)

F = Final State ($F \subseteq Q$)

An NFA is a theoretical Machine with no Restrictions such as processing all input alphabet for a particular state.

It Must be Converted to DFA in order for it to be Utilized, Consider NFA as a Rough Sketch of Machine

Q4 - What is an ENFA?

Ans - AN ENFA is a 5-Tuple FA Machine

$M = (Q, \Sigma, \delta, q_0, F)$

ϵ (Epsilon) String of length 0

PAGE NO.:

DATE:

FA : finite Automata

M = Name of the Machine

Q = non-empty finite Set

Σ = non-empty finite set of Input alphabet

$\delta = Q \times \Sigma \rightarrow Q$ (Transition State)

q_0 = Starting State $q_0 \in Q$

F = Final State $F \subseteq Q$

Just like NFA is an Theoretical Machine that is a Rough Sketch of Machine and must be converted to DFA in order to implement

NFA has no restrictions of Processing all the Input alphabet for a particular State.

Only difference B/w DFA & NFA is that NFA can accept ϵ (length of String 0) in short NFA can transit from one state to another even if ϵ is fed to it.

X — X — End of Theory X

So from here Numericals Start, All the ϵ (circle) ones are given By My Prof (Dr Samashethreddy) and non-circled ones are Both taken from last 5-years of TDC Syllabus (Non circle ones M1, M2, M3 attached in PDF)

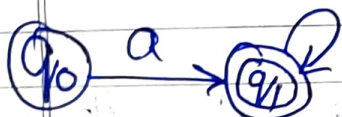
By the way My Prof said if we Read his Numericals (circled ones) that should be enough to score in Exams.

M.A., M.S., PDF do not Exist, So only My Sibs/My Notes

⇒ The Domm Numericals (210 of my Prof + Extra 0)

Prof (DFA) Design an DFA to accept strings of a 's having at least 1 a .

Ans - $\Sigma = a$
 $L = \{a, aa, aaa, \dots\}$



$M = (Q, \Sigma, \delta, q_0, F) \{ Q = \{q_0, q_1\} \}$

→ Steps of DFA

- (i) Identify the Input alphabets
- (ii) Identify the Minimal String
- (iii) Construct or Sketch DFA
- (iv) Identify Missing transition
- (v) Construct DFA using missing transition (Step 3 & 4)

δ	a
q_0	q_1
q_1	q_1

$\delta(q_0, a) \rightarrow q_1$
 $\delta(q_1, a) \rightarrow q_1$

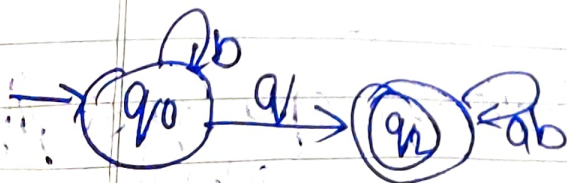
$F = \{q_1\} \subseteq Q$

Design an DFA with strings of a 's & b 's with at least 1 a .

Ans - $\Sigma = (a + b)$

$L = \{a, ab, abb, bab, \dots\}$

$\delta(q_0, a) \rightarrow q_1, \delta(q_0, b) \rightarrow q_0$



$\delta(q_1, a) \rightarrow q_2$
 $\delta(q_1, b) \rightarrow q_1$

	a	b
q_0	q_1	q_0
q_1	q_1	q_1

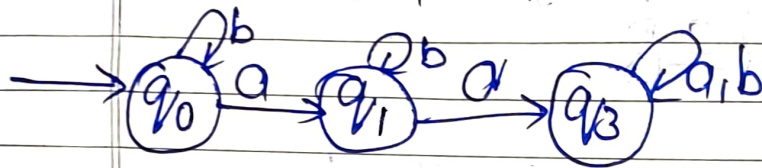
$$M = (Q, \Sigma, \delta, q_0, F)$$

$$\{q_1\} \subseteq Q$$

q_1 is the final state.

Ques Create a DFA to Accept String of a's & b's having exactly 1 a.

Ans $\Sigma = a, b$
 $L = \{a, ab, abb, abbb, baab, babb, \dots\}$

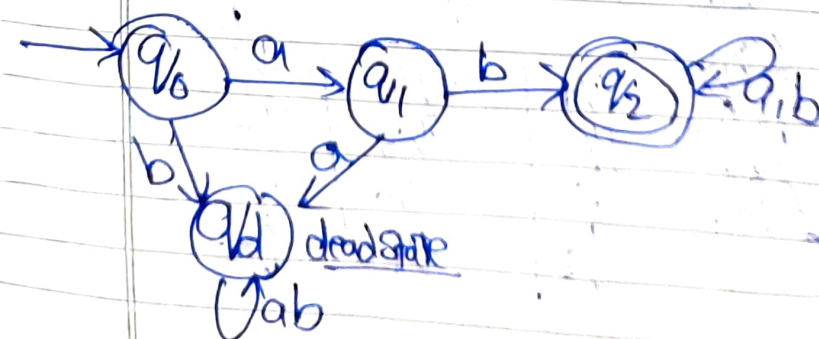


	a	b	
q_0	q_1	q_0	$\delta(q_0, a) \rightarrow q_1$
q_1	q_3	q_1	$\delta(q_0, b) \rightarrow q_0$
q_3	q_3	q_3	$\delta(q_1, a) \rightarrow q_3$
			$\delta(q_1, b) \rightarrow q_1$
			$\delta(q_3, a) \rightarrow q_3$
			$\delta(q_3, b) \rightarrow q_3$

$\{q_1\} \subseteq Q$ final state.

Ques Obtain the DFA to accept the a's & b's starting with ab

Ans - $\Sigma = a, b$
 $L = \{ab, aba, abb, \dots\}$



$$\delta(q_0, a) \rightarrow q_1$$

$$\delta(q_0, b) \rightarrow q_d$$

$$\delta(q_1, a) \rightarrow q_d$$

$$\delta(q_1, b) \rightarrow q_2$$

$$\delta(q_2, a) \rightarrow q_2$$

$$\delta(q_2, b) \rightarrow q_2$$

$$\delta(q_d, a) \rightarrow q_d$$

$$\delta(q_d, b) \rightarrow q_d$$

	a	b
q_0	q_1	q_d
q_1	q_d	q_2
q_2	q_2	q_2
q_d	q_d	q_d

Hence, $F = \{q_2\}$ final State.

$$\{q_2\} \subseteq Q$$

Tricky & Extra

Q5 Obtain DFA to Accept the language $L = w \in (a, b)^*$

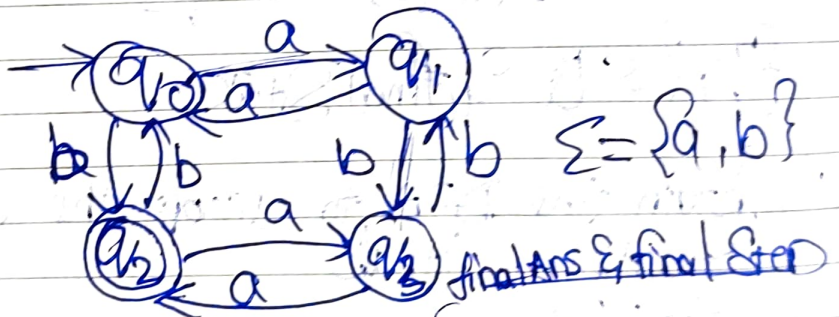
Ans w can have even no of a's, b's
odd no of a's, b's
Even no of a's, odd no of b's
Even no of b's, odd no of a's

(i) aabb

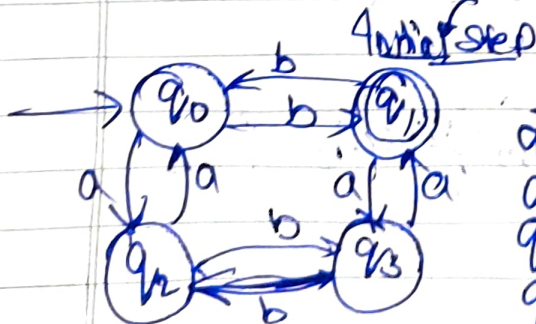
(ii) aaabbb

(iii) aab

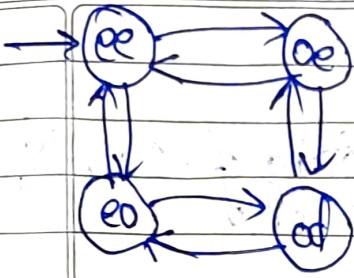
(iv) abb



Final Answer



	a	b
q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2



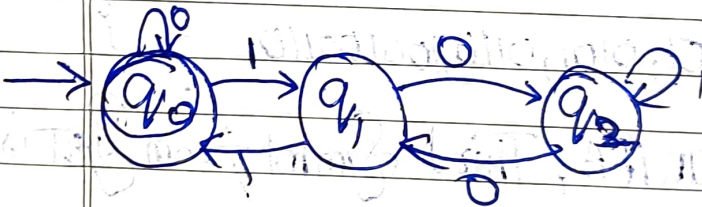
How logic works

even even, odd/even
even odd, odd/even

Q6) Design DFA to accept the strings to accept all binary strings which are divisible by 3.

Dec	Bin	Div
0	000	0
1	001	1
2	010	2
3	011	0
4	100	1
5	101	2
6	110	0

	1	0
q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_1	q_0



$$\delta(q_0, 0) \rightarrow q_0$$

$$\delta(q_0, 1) \rightarrow q_1$$

$$\delta(q_1, 0) \rightarrow q_0$$

$$\delta(q_1, 1) \rightarrow q_2$$

$$\delta(q_2, 0) \rightarrow q_1$$

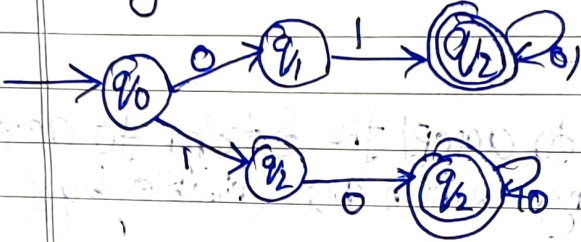
$$\delta(q_2, 1) \rightarrow q_0$$

$$F = \{q_0\}$$

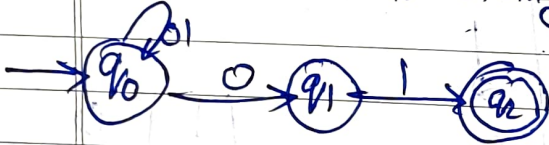
$$F \subseteq Q$$

NFA

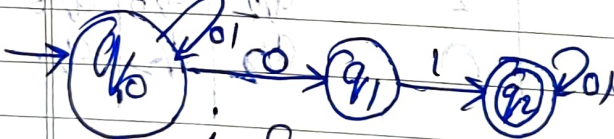
Q1. Construct NFA to accept the string of 0's & 1's, String starts with either 01 or 10



Q2. Construct NFA to string should start with 01

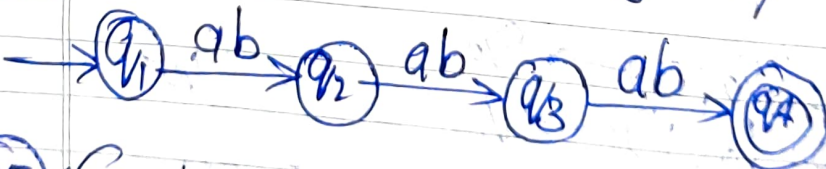


Q3. Substring 01 Construct an NFA



$L = \{01, 0101, 0110, 1001, 0011, 0110, \dots\}$

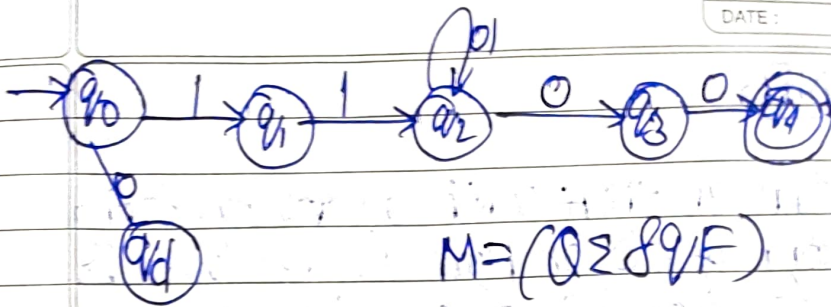
Q4. Construct an NFA, Third symbol from right is 'a' and 'b'



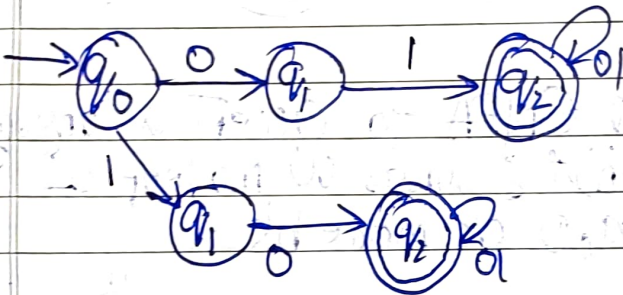
Q5. Construct an NFA, Third symbol does not end with 0



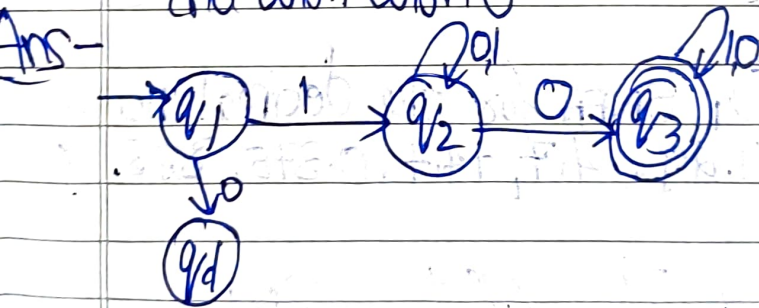
Q6. Construct an NFA to accept the string of 0's & 1's (100) starting with 11 & ending



Q77 - Construct an NFA to Accept the String of 0s & 1s. The String Contains either 01 or 010.

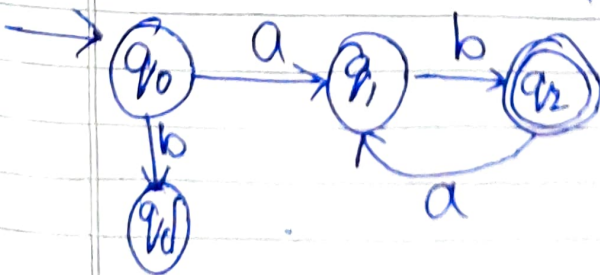


Q78 - Construct an NFA Starting with 1 & End with 0



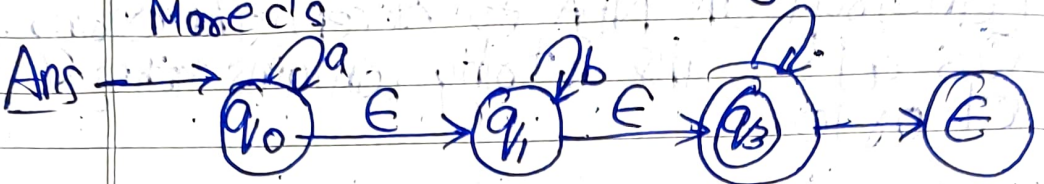
Q79 - $(aib)^n \mid n \geq 1$

$L = \{ab, abab, ababab, \dots\}$

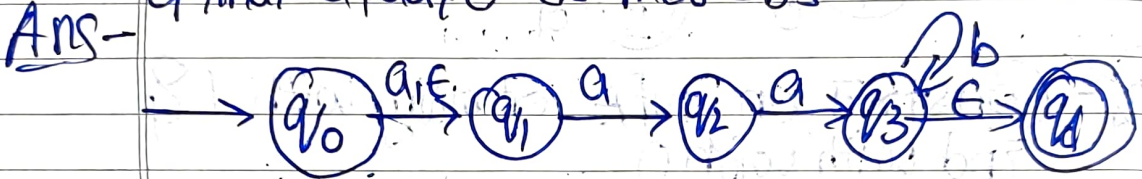


→ ENFA

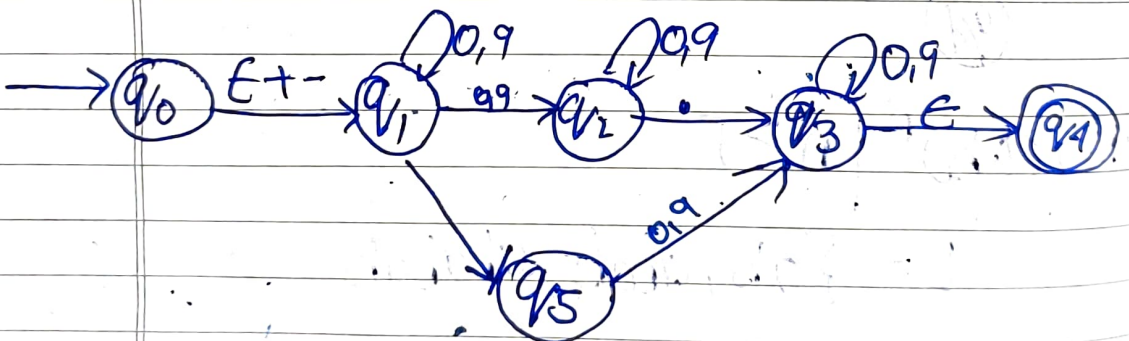
- ① Obtain an ENFA which accepts the string consisting of 0's or more 0's, followed by 0's or more b's, followed by 0's or more c's.



- ② Obtain the ENFA to accept a language $L = \{w \in (a,b)^*\}$ where w made up of optional a , aa , 0 or more b 's.

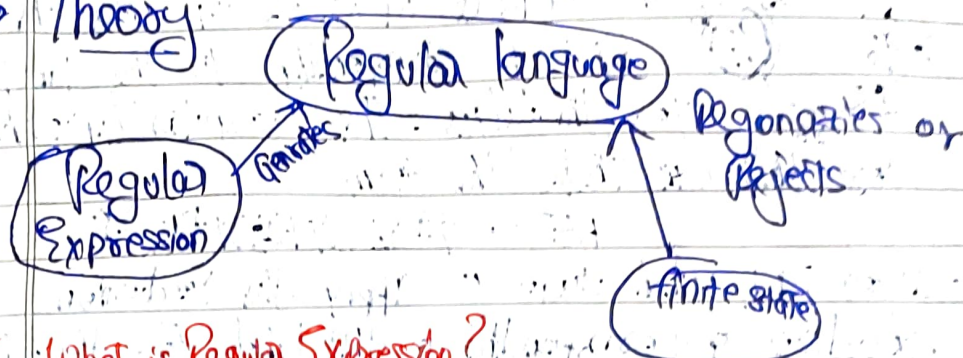


- 4mp ③ Design an ENFA for the decimal no $L = \{-47.29, 4.7, +4.7, 0.375, 375\}$.



Module 02 Regular Expression

→ Theory



Q1. What is Regular Expression?

Ans - Regular Expression means a Pattern or algorithm Expression which is useful to Represent Regular language. In order to Represent Regular language we use Regular Expression.

The language of FA {NFA, DFA, GNFA} is called Regular language.
Null String $\rightarrow \{ \epsilon \}$

- (i) if x_1 is Regular then x^* is also Regular
- (ii) if x_1 is Regular then x^+ is also Regular
- (iii) if x_1, x_2 is Regular then $(x_1 + x_2)$, $(x_1 \cdot x_2)$ is also Regular.

Q2. What are Regular Expression Identities?

Ans - (i) $\emptyset + R = R$

$$(ii) R\emptyset = \emptyset R = \emptyset$$

$$(iii) \epsilon \cdot R = R \cdot \epsilon = R$$

$$(iv) \epsilon^* = \epsilon \quad \& \quad \emptyset^* = \emptyset$$

$$(v) R + R = R$$

$$(vi) R^+ \cdot R^+ = R^+$$

$$(vii) R \cdot R^+ = R^+ \cdot R = R^+$$

$$(viii) (R^+)^+ = R^+$$

$$(ix) \epsilon + RR^+ = \epsilon + R^+R = R^+$$

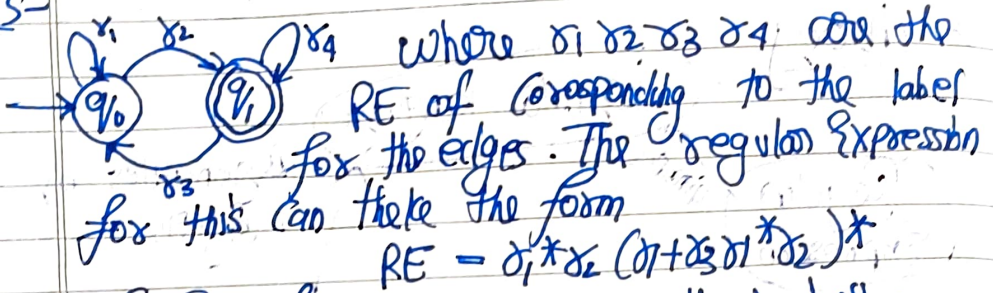
$$(x) P(QR)^+ = P(RQ)^+$$

$$(xi) (P + Q)^+ = (P^+ + Q^+)^+ = (P^+ \cdot Q^+)^+$$

$$(xii) (P + Q)R = PR + QR$$

Q3 To obtain RE from FA (By Terminating State)

Ans-



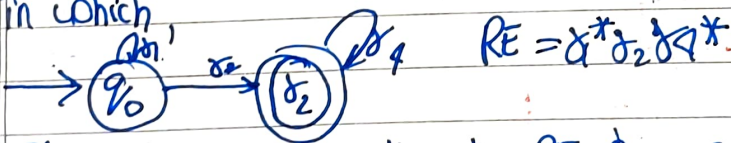
Note (1) For each final state q_i apply the reduction procedure & corresponding to the labels for the edges. The regular Expression for this can take the form

$$RE = x_1^* x_2 (x_3^* x_1^* x_2)^*$$

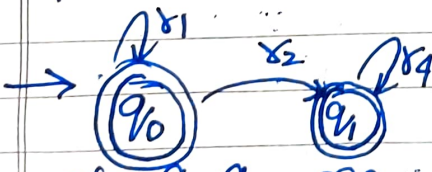
(2) If the starting state is also an accepting state, the state elimination process should be performed so that the final automata will be of the form,



(3) The final RE is the sum of regular Expression obtained from the reduced automata for each accepting string in which,



If x_3 is not there then the RE is as shown



If q_0, q_1 are both final state then RE of the form

$$RE = x_1^* + x_1^* x_2 x_4^*$$

For you

if you just need simple steps for your understanding

- step 1 - Initial state shouldn't have incoming edges. If Exist, create a new state and make it initial state.
- step 2 - FA should have a single final state. If Exist Multiple final state, create a new state make it a final state.
- step 3 - final state shouldn't have outgoing edges. If Exist, create a new final state and make it final.
- step 4 - Eliminate Every state step by step (no specific orders).
- step 5 - final FA should have only 2 state: initial & final. Cost upon initial & final state should be RE.

→ (q₀) Cost (RE) → (q₁)

Q. What's the difference Between DFA & NFA?

Ans - DFA | NFA

- | | |
|--|--|
| <p>(*) The Transition from a state is to single particular state for next state for each input symbol. Hence it is called Deterministic finite Automata (DFA).</p> <p>(*) Empty string transitions are not seen in DFA.</p> <p>(*) Requires more space.</p> <p>(*) Difficult to implement.</p> <p>(*) Complex to understand.</p> | <p>(*) The Transitions from a state can be Multiple next state for each input symbol. Hence it is called Non-Deterministic finite Automata.</p> <p>(*) Empty string transitions are permitted in NFA.</p> <p>(*) Requires less space.</p> <p>(*) Easy to implement.</p> <p>(*) Simple to understand.</p> |
|--|--|

Q. What is Pumping lemma?

Ans - The general strategy used to prove that certain language are not regular is shown below.

Step 1 - Assume the language L is regular and m be the finite no. of states of FA.

Step 2 - Select string x such that $|x| \geq n$ and break the string into substrings u, v and w such that

$x = uvw$ with constraints

- $v \neq \epsilon$ i.e. $|v| \geq 1$
- $|uv| \leq n$

Step 3 - And any i such that $uv^i w$ is not in L i.e. $uv^i w \notin L$

According to pumping lemma $uv^i w$ is the language for $i \geq 0$. So that the result is contradicting to the assumption the language is regular.

$\therefore$ The given language is not regular.

X — X — End of Theory X — X —

→ Numericals & Answers

Q1 Write RE for L accepting all combination of a 's except the null string over a set $\Sigma = a$

⇒ $RE = a^+$

Q2 Write the RE for L starting with a 's ends with any combination of b 's in ab^*

⇒ $RE = ab^*$

Q3 Write for L starting with a but not having consecutive b 's

⇒ $RE = (a|ab|)^*$

Q4 Write for L over Σ having even length of string

⇒ $L = \{\epsilon, 00, 11, 0000, 1111, \dots\}$

$RE = (00^*11^*)^*$

Q5 write for L having a string which should having atleast one 0 atleast one 1.

→ $\Sigma = ab$

$$(0+1)^* 0 (0+1)^* 1 (0+1)^* + (0+1)^* 1 (0+1)^* 0 (0+1)^*$$

$$RE = (0+10)^+ (0+1b)^+ (0+10)^+$$

Q6 obtain RE for string a's b's having length 2
→ $\Sigma = a, b$ $L = \{aa, ab, ba, bb\}$

$$RE = (a+b)^2$$

Q7 Obtain Regular Expression from string a's b's having length 2
→ $\Sigma = a, b$

$$L = \{aa, ab, ba, bb\}$$

$$RE = (a+b)^2$$

Q8 Obtain the Regular Expression language of length a, b length ≤ 2

Ans- $L = (\epsilon + a + b)(\epsilon + a + b)$
 $= (\epsilon + a + b)^2$

Q9 PT $a^n b^n$ where $n \geq 1$ is not a RE
 $L = \{ab, aabb, aaabbb, \dots\}$ $\Sigma = a, b$

$$S = aabb$$

$$|xy| \leq P \quad 3 \leq 4$$

$$|y| > 1 \quad 2 > 1$$

$$xy^iz$$

$$ab \rightarrow i=0$$

$$aabb \rightarrow i=0$$

$$aaabbb \rightarrow i=2$$

$\therefore L$ is not a RE

$$\text{let } n=4$$

$$\text{let } z = aabb$$

$$|z| = 4 \geq 4$$

$$\frac{a}{u} \frac{ab}{v} \frac{b}{w} \quad |a \cdot ab| \leq n$$

$$3 \leq 4 \text{ True}$$

$$|u| \geq |v|$$

$$|v| \geq |w|$$

$$2 \geq 1 \text{ True}$$

$$a \cdot (ab)^0 \cdot b \Rightarrow ab \in N$$

$$a \cdot (ab)^P \cdot b \text{ Not True Hence not RE}$$

⑩ Obtain Regular Expression for String of length ≤ 10

$$\Sigma = ab$$

$$RE = (\epsilon + a + b) + (\epsilon + a + b) + \dots + (\epsilon + a + b)$$

$$= (\epsilon + a + b)^{10}$$

⑪ obtain RE of String of a's b's having even length

$$\Sigma = ab$$

$$RE = (a+b)(a+b)^*$$

⑫ Obtain RE of String a's b's having odd no of length $\Sigma = ab$

$$RE = (a+b)(aa+bb+ab+ba)^*$$

⑬ Write a RE for accepting all the String

Containing any no of a's & b's

$$L = \{\epsilon, a, b, ab, ba, bb, \dots\}$$

$$RE = (a+b)^*$$

⑭ Design a RE which ends with 00 over ab

$$\Sigma = ab \rightarrow \Sigma = 0, 1$$

$$RE = (0+1)^*00$$

$$L = \{00, 000, 100, 1100, 111000, \dots\}$$

⑮ Design a RE that starts with 1 and ends with 0

$$\Sigma = 0, 1$$

$$L = \{10, 100, 1000, 10000, \dots\}$$

$$RE = 1(1+0)^*0$$

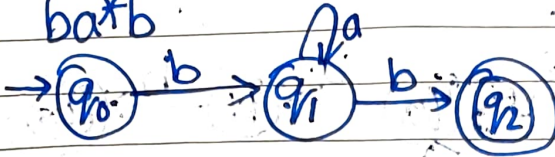
⑯ Design an RE for n a's and n b's followed by n c's

$$\Rightarrow RE = a^n b^n c^n$$

(17) Convert to RE

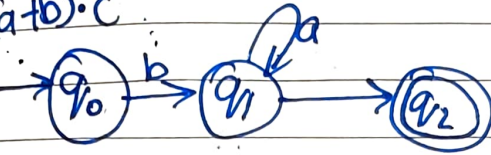
(a)

ba^*b



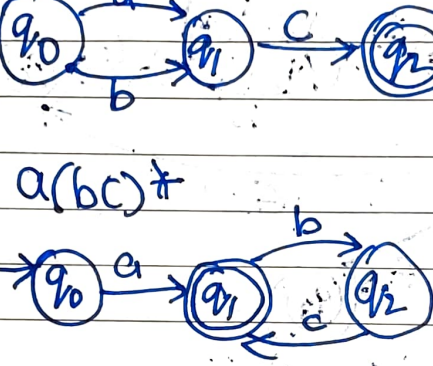
(b)

$(a+b) \cdot c$

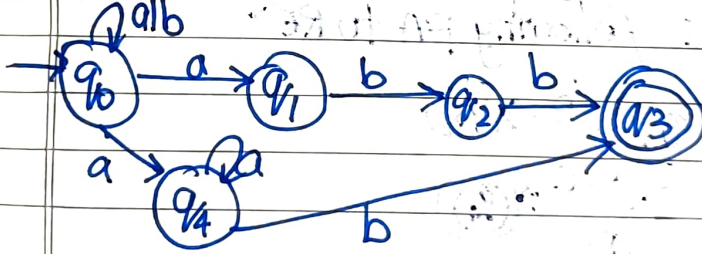


(c)

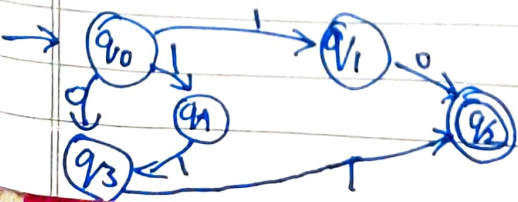
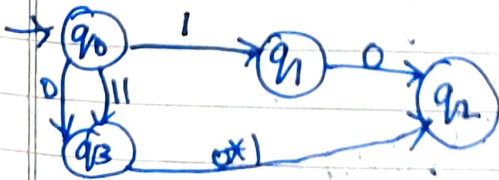
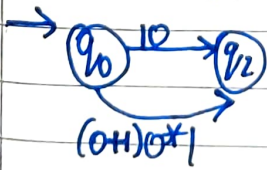
$a(bc)^*$



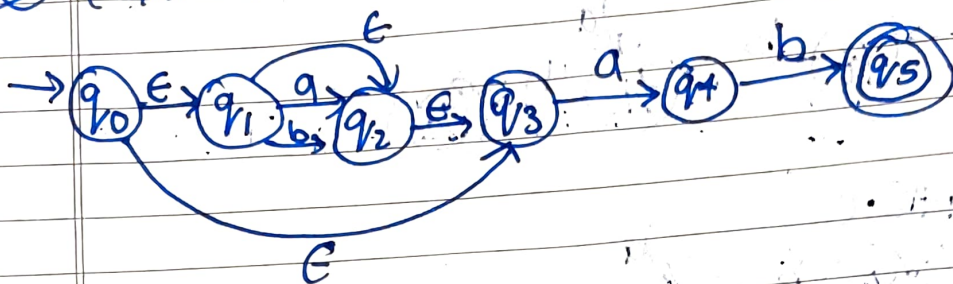
(18) Convert RE to DA $(a|b)^*$ or $(a+b)^*$ $(abb|a^*b)^*$



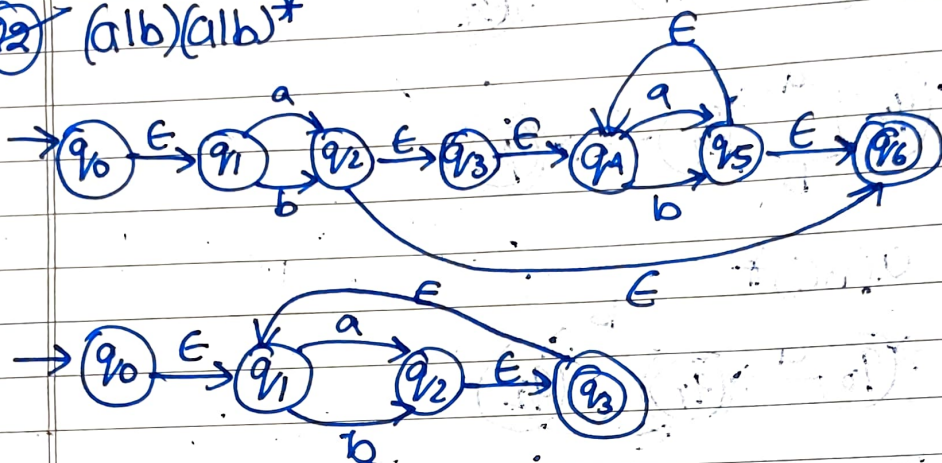
(19) $(0 + (0+11)^*)^*$



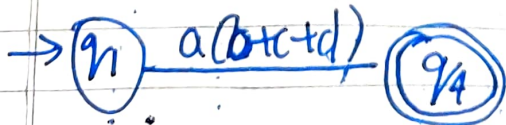
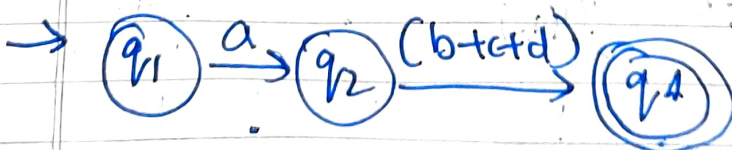
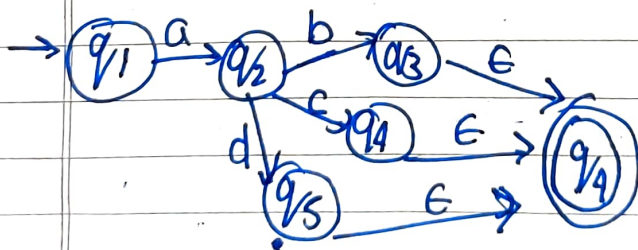
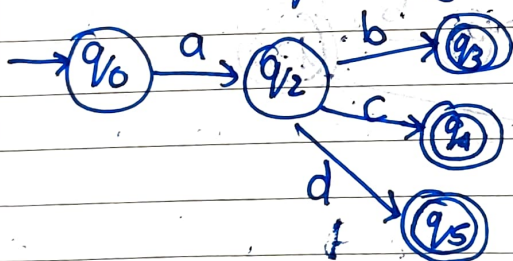
~~21~~ $(ab)^*a.b$



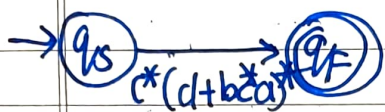
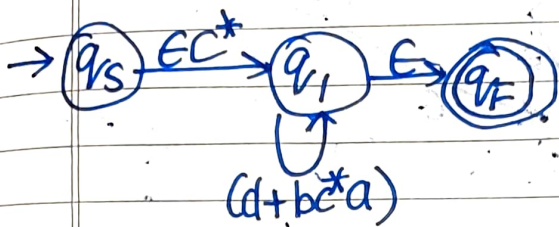
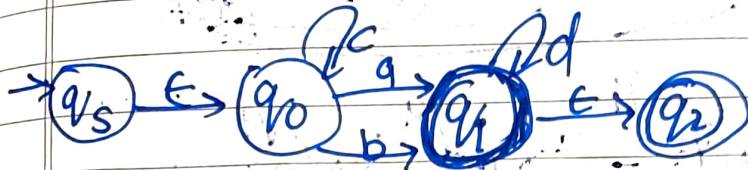
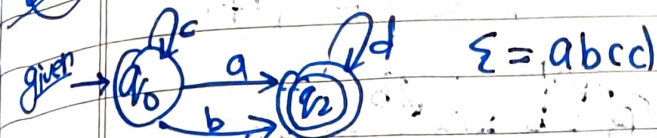
~~22~~ $(ab)(ab)^*$



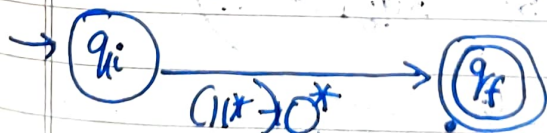
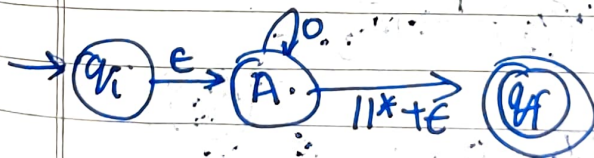
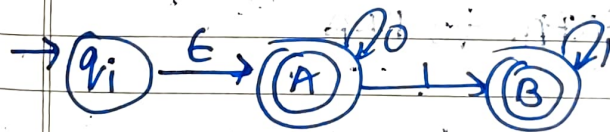
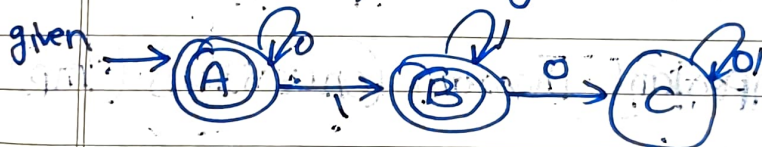
~~23~~ Convert the following FA to RE



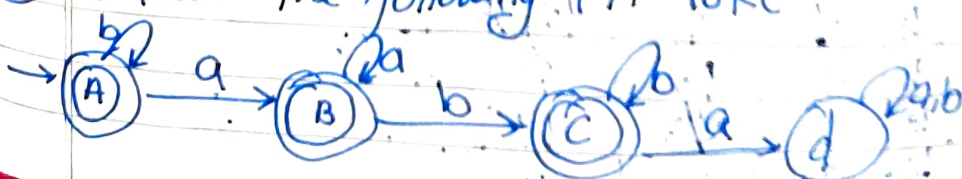
Q4) q_1 is the starting state, q_2 is the final state

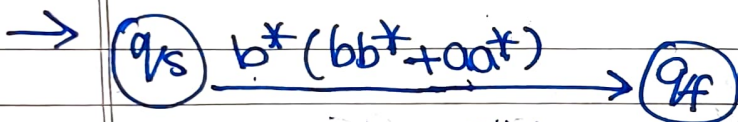
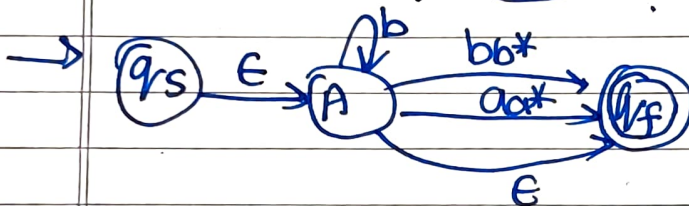
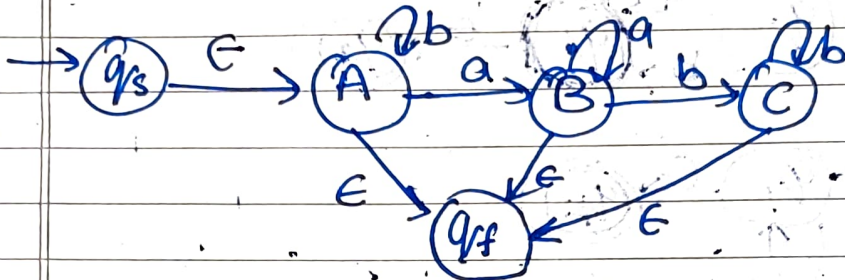
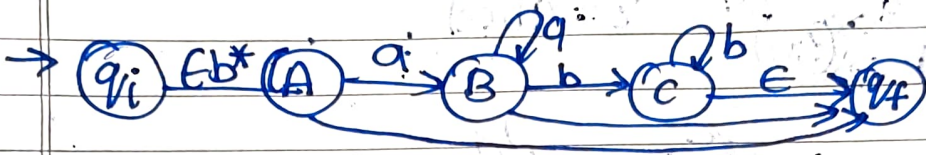
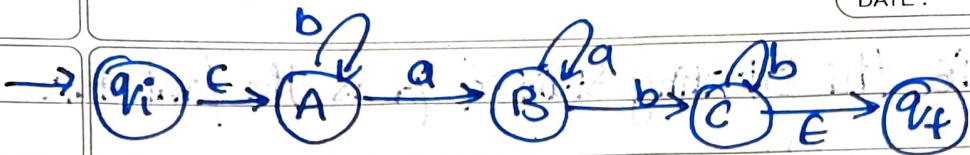


Q5) Convert the following FA to RE:



Q6) Convert the following FA to RE:

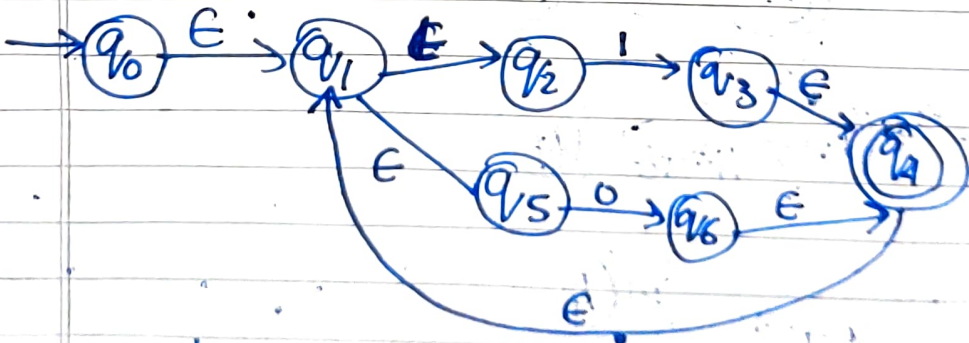




✗ END of Numericals ✗

*→ Some Important Theorem Conversions (Imp)

I Conversion of ENFA to DFA



ENFA TABLE	0	1	ε
q ₀	φ	φ	q ₁
q ₁	φ	φ	q ₂ , q ₅
q ₂	φ	q ₃	φ
q ₃	φ	φ	q ₄
q ₄	q ₆	φ	q ₁

	0	1	ϵ
q_5	ϕ	ϕ	ϕ
q_6	ϕ	ϕ	q_1

$$\Rightarrow E_{\text{closure}} = \{q_0, q_1, q_2, q_5\}$$

$$q_1 = E(q_1) = \{q_1, q_2, q_5, q_0\}$$

$$q_2 = E(q_2) = \{q_1, q_2, q_5\}$$

$$q_3 = E(q_3) = \{q_1, q_3\}$$

$$q_3 = E(q_3) = \{q_3, q_4, q_1, q_2, q_5\}$$

$$q_4 = E(q_4) = \{q_4, q_1, q_2, q_5\}$$

$$q_5 = E(q_5) = \{q_5\}$$

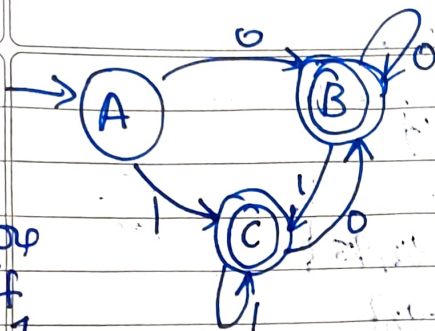
$$q_6 = E(q_6) = \{q_6, q_4, q_1, q_2, q_5\}$$

$\Rightarrow$ DFA Table

(A)	(B)	(C)
$\{q_0, q_1, q_2, q_5\}$	$\{q_6, q_4, q_1, q_2, q_5\}$	$\{q_3, q_4, q_1, q_2, q_5\}$
$\{q_6, q_4, q_1, q_2, q_5\}$	$\{q_6, q_4, q_1, q_2, q_5\}$	$\{q_3, q_4, q_1, q_2, q_5\}$
$\{q_3, q_4, q_1, q_2, q_5\}$	$\{q_6, q_4, q_1, q_2, q_5\}$	$\{q_3, q_4, q_1, q_2, q_5\}$

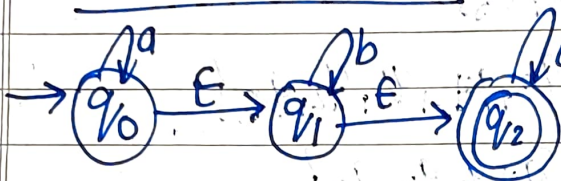
Processed all States A, B, C
after ABC States Repeat So No Need to Write.

Use formula $E_{\text{clos}}(S(A;0))$
always start with q_0



Till how
Portions of
Internal 1

* → Convert ENFA to NFA



	a	b	c	
q ₀	q ₀	∅	∅	q ₀
q ₁	∅	q ₁	∅	q ₁
q ₂	∅	∅	q ₂	∅

ENFA Table

$$\left. \begin{aligned} \text{Ecl}_{\text{os}}(q_0) &= \{q_0, q_1, q_2\} \\ \text{Ecl}_{\text{os}}(q_1) &= \{q_1, q_2\} \\ \text{Ecl}_{\text{os}}(q_2) &= \{q_2\} \end{aligned} \right\} \text{NFA}$$

→

	a	b	c
q ₀	{q ₀ , q ₁ , q ₂ }	{q ₁ , q ₂ }	{q ₂ }
q ₁	∅	{q ₁ , q ₂ }	{q ₂ }
q ₂	∅	∅	{q ₂ }

NFA Table

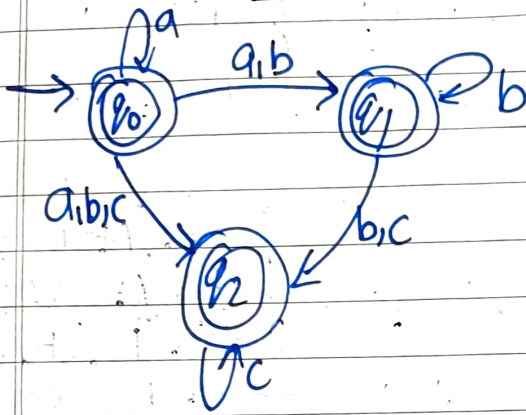
→ Ecl_{os} elements in diagram

→ Calculation By formula for q₀ (do same for q₁)

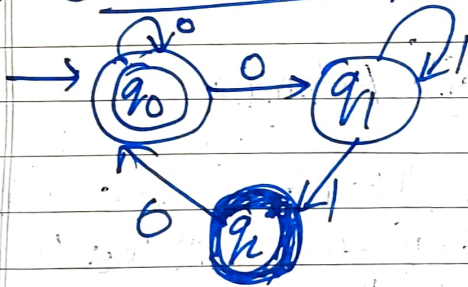
$$\begin{aligned} \# \text{Ecl}_{\text{os}}(\delta(q_0, a)) &= (\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)) \\ &= q_0 \Rightarrow \{q_0, q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \# \text{Ecl}_{\text{os}}(\delta(q_1, b)) &= \text{Ecl}_{\text{os}}(\delta(q_0, b) \cup \delta(q_2, b) \cup \delta(q_2, b)) \\ &= \text{Ecl}_{\text{os}}(\emptyset \cup q_2 \cup \emptyset) \Rightarrow \{q_2\} \Rightarrow \{q_1, q_2\} \end{aligned}$$

$$\begin{aligned}
 \# \quad & \text{Eclos}(S(q_0, q_1, q_2), c) \\
 &= \text{Eclos}(S(q_0, c) \cup S(q_1, c) \cup S(q_2, c)) \\
 &= (\phi \cup \phi \cup q_2) \\
 &= \underline{\underline{q_2}}
 \end{aligned}$$



↔ Convert NFA to DFA

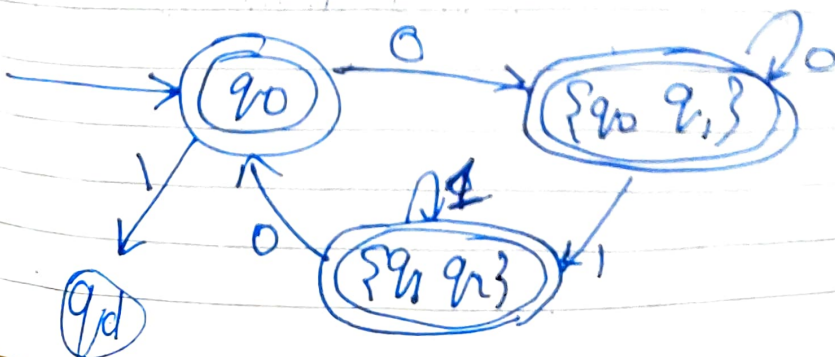


NFA	0	1
q_0	$\{q_0, q_1\}$	ϕ
q_1	ϕ	$\{q_1, q_2\}$
q_2	$\{q_0\}$	ϕ

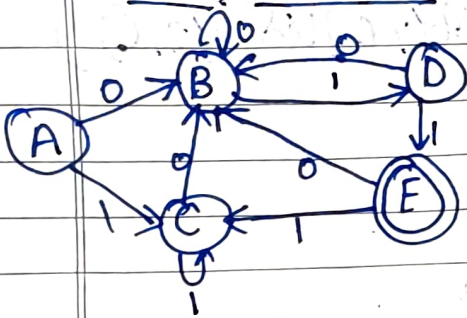
Now DFA

q	0	1
q_0	$\{q_0, q_1\}$	$\{q_d\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
q_2	$\{q_0\}$	$\{q_1, q_2\}$

$$\begin{aligned}
 &= \delta(q_0, q_1, 0) \\
 &= \delta(q_0, 0) \cup \delta(q_1, 0) \\
 &= \underline{\underline{\{q_0, q_1\}}}
 \end{aligned}$$

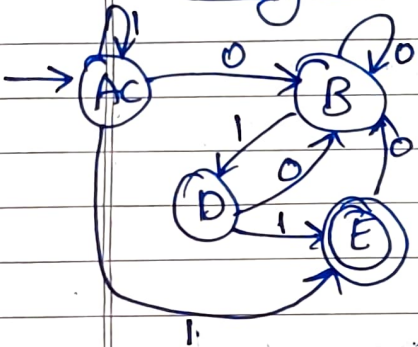


→ Minimization of DFA



δ	0	1
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

Minimizing DFA



- 0 - EQ (ABCD) {E}
- 1 - EQ (ABC) {D} {E}
- 2 - EQ (AC) {B} {D} {E}
- 3 - EQ (AC) {B} {D} {E}

→ Limitations of Finite Automata

1. There exist so many problems for which we cannot construct FA & still we want some solution to solve these problem Ex -

(*) Check for Matching Parenthesis is not possible

(*) Counting the Number of a's & b's not possible using FA so it cannot be used as a counter

(*) has a limited amount of memory